Docker
Sudo apt update
Sudo apt install docker.io (installs docker daemon and docker cli)
sudo systemctl start docker (starts Docker daemon immediately)
sudo systemctl enable docker (enable on boot)
sudo usermod -aG docker $USER (docker is the group that can run docker commands, now $USER=currently logged in user)

```
hekhek@hehek:~$ sudo usermod -aG docker $USER
hekhek@hehek:~$ whoami
hekhek
```

(by default,docker needs sudo; now- only docker) BUT :

```
hekhek@hehek:~$ docker run hello-world
docker: permission denied while trying to connect to the Docker daemon socket at unix:///var/run/docker.sock: Head "http://%2Fvar%2Frun%2Fdocker.sock/_ping": dial unix /var/run/docker.sock: connect: permission denied

Run 'docker run --help' for more information
hekhek@hehek:~$ sudo docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
Digest: sha256:452a468a4bf985040037cb6d5392410206e47db9bf5b7278d281f94d1c2d0931
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/
```

??
The **newgrp** command is used to change the current GID (group ID) during a login session. If a hyphen ("**-**") is included as an argument, the user's environment is initialized as though he or she had logged in; otherwise, the current working environment remains unchanged. **newgrp** changes the current real group ID to the specified *group*, or, if no *group* is specified, to the default group listed in the file **/etc/passwd**. **newgrp** also tries to add the group to the user groupset.

it changes your current session so that the **docker group becomes active immediately**, without requiring a logout and login.

In Linux, when a user is added to a new group using `usermod -aG`, the change is written to system files like `/etc/group`, but it does not affect already running sessions. This means your current shell still uses the old group list. The `newgrp` command forces the system to reload group membership for that session by spawning a new shell where the specified group is included in your active group set.

NOW ;

```
hekhek@hehek:~$ sudo usermod -aG docker $USER
hekhek@hehek:~$ newgrp docker
hekhek@hehek:~$ docker run hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/
```

Website :

```
hekhek@hehek:~$ mkdir simplesite
hekhek@hehek:~$ cd simplesite
hekhek@hehek:~/simplesite$ nano index.html
```

Nginx (pronounced "engine x"[9] / ˌɛndʒɪnˈɛks/ *EN-jin-EKS*, stylized as **NGINX** or **nginx**) is a web server that can also be used as a reverse proxy, load balancer, mail proxy and HTTP cache. The software was created by Russian developer Igor Sysoev and publicly released in 2004.[10] Nginx is free and open-source software, released under the terms of the 2-clause BSD license. A large fraction of web servers use Nginx,[11] often as a load balancer.[12]

```
Common Commands:
  run         Create and run a new container from an image
  exec        Execute a command in a running container
  ps          List containers
  build       Build an image from a Dockerfile
  pull        Download an image from a registry
  push        Upload an image to a registry
  images      List images
  login       Authenticate to a registry
  logout      Log out from a registry
  search      Search Docker Hub for images
  version     Show the Docker version information
  info        Display system-wide information

Management Commands:
  builder     Manage builds
  container   Manage containers
  context     Manage contexts
  image       Manage images
  manifest    Manage Docker image manifests and manifest lists
  network     Manage networks
  plugin      Manage plugins
  system      Manage Docker
  trust       Manage trust on Docker images
```

docker run -d -p 8080:80 -v $(pwd):/usr/share/nginx/html nginx

`-d :` Runs container in background (detached mode) ,Without this → terminal will be blocked

`-p 8080:80 :` Maps ports: host:8080 → container:80 (host_port : container_port)

`-v $(pwd):/usr/share/nginx/html`

- `-v` → volume mount
- `$(pwd)` → current directory ; **`$(pwd)`**: This is a shortcut for "Print Working Directory." It grabs the full path of the folder you are currently sitting in.
- `/usr/share/nginx/html` → Nginx web root ; **`/usr/share/nginx/html`**: This is the specific "magic folder" inside the Nginx container where it looks for an `index.html` file to show the world.

https://docs.docker.com/engine/network/port-publishing/

Flag value	Description
`-p 8080:80`	Map port `8080` on the Docker host to TCP port `80` in the container.
`-p 192.168.1.100:8080:80`	Map port `8080` on the Docker host IP `192.168.1.100` to TCP port `80` in the container.
`-p 8080:80/udp`	Map port `8080` on the Docker host to UDP port `80` in the container.
`-p 8080:80/tcp -p 8080:80/udp`	Map TCP port `8080` on the Docker host to TCP port `80` in the container, and map UDP port `8080` on the Docker host to UDP port `80` in the container.

The `nginx` Docker image already contains:

- Nginx installed
- Configuration files
- Default settings

One of those settings is: listen 80;

So, we started Nginx using nginx image inside docker

```
hekhek@hehek:~/simplesite$ docker run -d -p 8080:80 -v $(pwd):/usr/share/nginx/html nginx
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
ec781dee3f47: Pull complete
bb3d0aa29654: Pull complete
510ddf6557d6: Pull complete
cde7a05ae428: Pull complete
587e3d84dbb5: Pull complete
3189680c601f: Pull complete
5e815e07e569: Pull complete
Digest: sha256:7150b3a39203cb5bee612ff4a9d18774f8c7caf6399d6e8985e97e28eb751c18
Status: Downloaded newer image for nginx:latest
1ad52db45b35faf1f7fc5a8129b9e390e50f52856460065cac0529ded54ef28b
```

Now, let's see docker run nginx :

hekhek@hehek:~$ docker run nginx

- /docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration [Before Nginx actually starts serving web pages, it runs a series of helper scripts located in `/docker-entrypoint.d/`]
- /docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
- /docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
- 10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
- 10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
- /docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
- /docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
- /docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
- /docker-entrypoint.sh: Configuration complete; ready for start up

- 2026/03/30 13:41:03 [notice] 1#1: using the "epoll" event method
- 2026/03/30 13:41:03 [notice] 1#1: nginx/1.29.7
- 2026/03/30 13:41:03 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
- 2026/03/30 13:41:03 [notice] 1#1: OS: Linux 6.8.0-1051-azure
- 2026/03/30 13:41:03 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
- 2026/03/30 13:41:03 [notice] 1#1: start worker processes
- 2026/03/30 13:41:03 [notice] 1#1: start worker process 29
- 2026/03/30 13:41:03 [notice] 1#1: start worker process 30
- ^C2026/03/30 13:41:16 [notice] 1#1: signal 2 (SIGINT) received, exiting [By hitting **Ctrl+C**, you sent a **SIGINT** (Signal Interrupt) to the container.]
- 2026/03/30 13:41:16 [notice] 29#29: exiting
- 2026/03/30 13:41:16 [notice] 30#30: exiting
- 2026/03/30 13:41:16 [notice] 29#29: exit
- 2026/03/30 13:41:16 [notice] 30#30: exit
- 2026/03/30 13:41:16 [notice] 1#1: signal 17 (SIGCHLD) received from 29
- 2026/03/30 13:41:16 [notice] 1#1: worker process 29 exited with code 0
- 2026/03/30 13:41:16 [notice] 1#1: worker process 30 exited with code 0

  [Once the setup was finished, the actual Nginx master process started.

  The Master (PID 1): This is the "boss" process. In Docker, if this process stops, the container dies.

  The Workers (PID 29 & 30): These are the "employees" that handle actual web traffic. You can see Nginx started two of them because your environment likely has two CPU cores.]

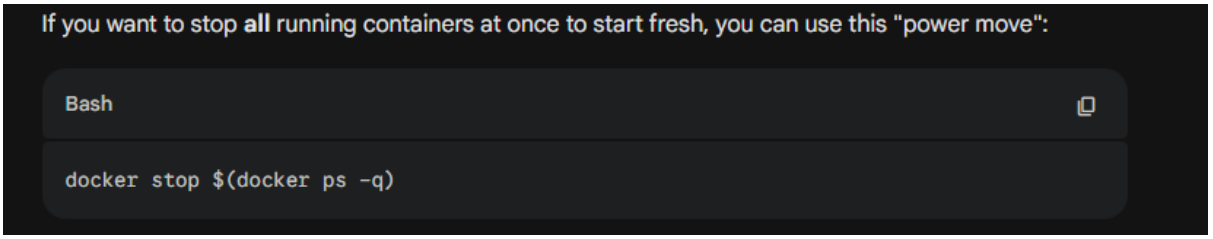
- 2026/03/30 13:41:16 [notice] 1#1: exit

To find running container : docker ps

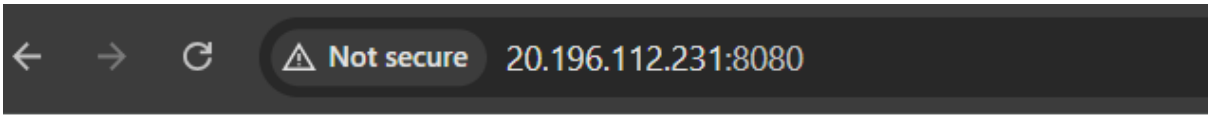
hekhek@hehek:~/simplesite$ docker ps -q

1ad52db45b35

(Just gives id)



Now, after curl ifconfig.me -> i got my vm's IP , used it in my machine and added inbound rule in azure for port 8080 : then =>



Namaskkaram Friend

This is my simple website

WOWWW…!!

- When i opened 20.196.112.231:8080–**browser sent a request** to VM
- The request reached **port 8080 on this VM**
- Docker forwarded it to **port 80 inside the container**
- Nginx received it and looked inside:

/usr/share/nginx/html

- It found `index.html`
- It sent that file back to the browser

```
hekhek@hehek:~/simplesite$ docker stop 1ad52db45b35
1ad52db45b35
hekhek@hehek:~/simplesite$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
hekhek@hehek:~/simplesite$ docker run -d --restart=always -p 8080:80 -v $(pwd
/share/nginx/html nginx
bf1f9da01540c409a958a5ecc6578be05325b0fa4fed235914526b7a836bad00
hekhek@hehek:~/simplesite$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED          STATUS
PORTS                                   NAMES
bf1f9da01540   nginx     "/docker-entrypoint.…"   17 seconds ago   Up 16 seco
0.0.0.0:8080->80/tcp, [::]:8080->80/tcp   cool_noyce
hekhek@hehek:~/simplesite$
```

docker run -d --restart=always -p 8080:80 -v $(pwd):/usr/share/nginx/html nginx

`--restart=always`**The Safety Net.** If the container crashes or your entire computer restarts, Docker will automatically boot this container back up. What is the change now?

Scenario	Default (No Flag)	With `--restart=always`
Manual Stop	Container stays stopped.	Container stays stopped (until you manually start it or Docker restarts).
App Crash	Container exits and stays dead.	**Docker immediately restarts it.**
System Reboot	Container stays stopped after boot.	**Container starts automatically on boot.**
Docker Daemon Restart	Container stays stopped.	**Container starts automatically.**

On Linux, traditionally, a process could either run as root, and so have limitless access to the system, or as a non-root user, subject to a set of restrictions. Since version 2.2 of the kernel, capabilities were introduced as a way to grant permissions in a more granular way.Not exactly *every* power, but:

- Docker gives a **default set of capabilities**

If we do: --cap-add=NET_BIND_SERVICE

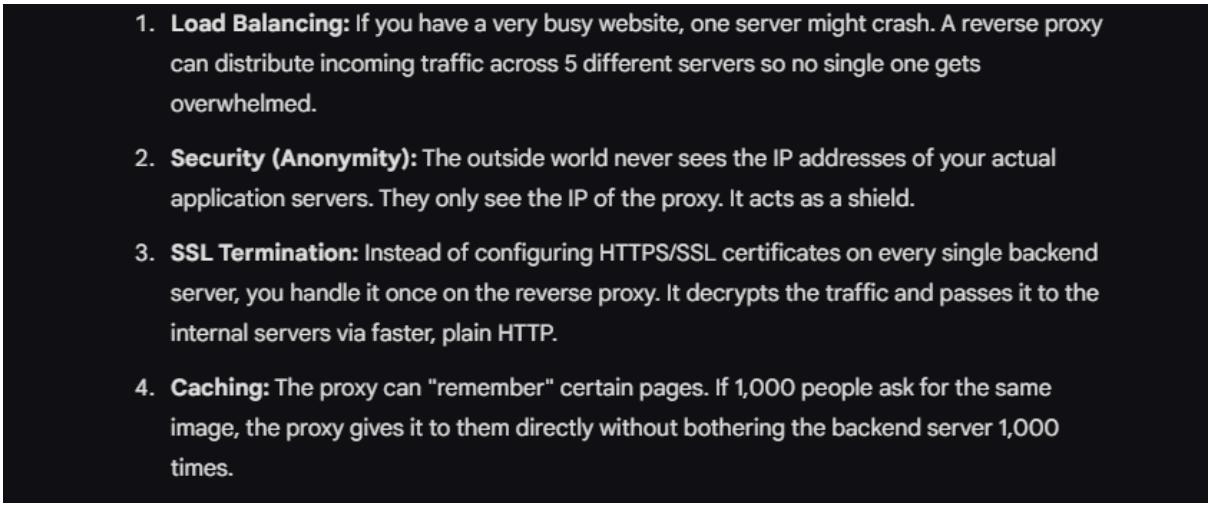
Container now has:

- default capabilities + `NET_BIND_SERVICE`

The use of Linux capabilities was not required for this setup because the application is a simple static web server that does not require privileged operations. The container runs with Docker's default capability set, which is sufficient for serving web content. Additional capabilities were not granted to avoid unnecessary elevation of privileges and to maintain a simpler configuration.

Reverse proxy :

In a standard web setup, a client (browser) communicates directly with a web server. In a reverse proxy setup, the client communicates with the proxy, which then communicates with one or more "backend" servers.



Forward Proxy (e.g., a VPN): Sits in front of the **Client**. It hides who *you* are from the internet.

Reverse Proxy (e.g., Nginx): Sits in front of the **Server**. It hides the *server's* internal structure from the internet.

- Reverse proxy is a system where a server (Nginx on the host) receives incoming requests and forwards them to backend services (your Docker containers). Instead of users directly accessing services using IP:port, the reverse proxy provides a single entry point (IP:80) and internally routes traffic to the correct container.
- I created a simple static website using an `index.html` file. This file acts as the default entry point that a web server serves when a directory is requested.
- And ran a Docker container using:
  `docker run -d -p 8080:80 -v $(pwd):/usr/share/nginx/html nginx`
- This means a container was started in detached mode, port 8080 on the host was mapped to port 80 inside the container, and current directory was mounted into the container's web root directory. Nginx inside the container serves files from `/usr/share/nginx/html`, so our local HTML file became accessible through the container.
- Since the nginx image was not available locally, Docker pulled it from Docker Hub. The image contains a pre-configured Nginx server that listens on port 80 inside the container.
- When accessing `http://IP:8080`, the request flows as:
  browser → VM IP:8080 → Docker port mapping → container port 80 → Nginx inside container → index.html → response back to browser.
- Then we used `--restart=always` to ensure the container automatically restarts after a system reboot or if it stops. This makes the service persistent.
- Then explored container management using `docker ps` to list running containers. Using `docker ps -q` gives only container IDs, which can be used with commands like `docker stop <id>` and `docker start <id>` to manage containers.
- A second website in another directory and ran a second container mapped to port 8081. At this stage:
  site1 → port 8080
  site2 → port 8081
- Both were accessible independently using IP:8080 and IP:8081.
- To avoid exposing ports and to simulate a real-world setup, installed Nginx on the host and configured it as a reverse proxy. Instead of users accessing

different ports, they access a single endpoint (IP:80), and Nginx routes traffic internally.

- Now modified the Nginx configuration file to add routing rules:
  requests to `/site1/` are forwarded to `localhost:8080/`
  requests to `/site2/` are forwarded to `localhost:8081/`
- This changed the flow to:
  browser → VM IP:80 → host Nginx → proxy_pass → appropriate container → response.
- wealso enabled firewall access using UFW (`ufw allow 80`) and configured Azure inbound rules to allow port 80. Without this, external traffic cannot reach Nginx even if it is correctly configured.
- I attempted to use a file named `indexi.html`. This failed because Nginx (inside the container) looks for default files like `index.html`. Since it could not find a valid index file and directory listing is disabled, it returned a 403 Forbidden error. The change that made in the host Nginx config (`index indexi.html`) had no effect because file serving is handled by the Nginx inside the container, not the host Nginx.
- The 403 Forbidden error means the server understood the request but refused to serve the content. In this case, it occurred because there was no valid default file (`index.html`) for the requested directory.
- Initially considered permissions as a cause. While permissions can cause 403 errors when the container cannot read files, in our case the primary issue was the absence of a valid index file. Renaming `indexi.html` to `index.html` resolved the issue because it matched the default file expected by Nginx.
- If we want to use a different filename such as `abc.html`, we must explicitly request it using `http://IP/site2/abc.html`, or modify the Nginx configuration inside the container to change the default index directive. Without that, Nginx will not automatically serve it.
- Overall flow of final system:
  user sends request to VM IP
  request reaches port 80
  host Nginx receives request
  based on URL path, it forwards request to correct container
  container Nginx serves HTML file
  response is sent back to user

- This setup demonstrates how reverse proxy abstracts backend services, hides internal ports, and enables multiple applications to run behind a single public interface.

```
hekhek@hehek:~/simplesite2$ curl http://localhost:8081
<html>
<head><title>403 Forbidden</title></head>
<body>
<center><h1>403 Forbidden</h1></center>
<hr><center>nginx/1.29.7</center>
</body>
</html>
hekhek@hehek:~/simplesite2$ chmod -R 755 ~/simplesite2
hekhek@hehek:~/simplesite2$ docker restart exciting_newton
exciting_newton
hekhek@hehek:~/simplesite2$ sudo systemctl restart nginx
hekhek@hehek:~/simplesite2$ curl http://localhost:8081
<html>
<head><title>403 Forbidden</title></head>
<body>
<center><h1>403 Forbidden</h1></center>
<hr><center>nginx/1.29.7</center>
</body>
</html>
```

```
hekhek@hehek:~/simplesite2$ mv indexi.html index.html
hekhek@hehek:~/simplesite2$ curl http://localhost:8081
<h1>This is Site 2</h1>
```

```
hekhek@hehek:~/simplesite2$ chmod 755 ~/simplesite
chmod 644 ~/simplesite/index.html

chmod 755 ~/simplesite2
chmod 644 ~/simplesite2/index.html
hekhek@hehek:~/simplesite2$ ls -l
total 4
-rw-r--r-- 1 hekhek hekhek 24 Mar 30 15:31 index.html
hekhek@hehek:~/simplesite2$ cd ..
hekhek@hehek:~$ ls -l
total 8
drwxr-xr-x 2 hekhek docker 4096 Mar 30 13:08 simplesite
drwxr-xr-x 2 hekhek hekhek 4096 Mar 30 16:07 simplesite2
```

Directory permissions=766 and file permissions=644